

Notes (Week 10 Wednesday)

These are the notes I wrote during the lecture. Most of this is going over Sample Exam 2 from the sidebar.

The exam is open-book,
 – you can use any **passive** resource,
 but **active** resources are limited.

You can read the course website,
 your favourite book,
 your favourite website

You can't actively solicit answers,
 don't ask on Chegg, stackoverflowm
 ...

Exception:
 – You can ask on Ed, if you make the thread initially private.

Q1

```
0 = λf x. x
1 = λf x. f x
2 = λf x. f(f x)
```

```
true = λt f. t
false = λt f. f
```

A: $\lambda f x. x$
 represents both 0 and false.

Note: we don't differentiate
 between ** α -equivalent** terms

α -equivalence means

"equal up to choice of
 placeholders for bound
 variables"

Q2

CNF: a formula that is a
 conjunction of disjunction
 of atoms:

```
( _ v _ ) ∧ ( _ v _ v _ )
```

An atom is a variable,

or a negated variable

For CNF:

for each row where ϕ is false,
write a disjunction that captures
"how do we avoid this line"

$(A \vee B \vee \neg C) \wedge$
 $(A \vee \neg B \vee \neg C) \wedge$
 $(\neg A \vee B \vee C) \wedge$
 $(\neg A \vee \neg B \vee C)$

^ Done!

Remember: the CNF is not necessarily
unique, we could present
other variants.

Q3

Prove $(R^*)^* \subseteq R^*$

We have an inductively defined
relation R^* , and that means
we can use structural induction
over the inference rules.

Here's the induction principle:

If $x R^* y$, and if:
 Base cases: $\forall x. \phi(x, x)$
 and
 Inductive: $\forall x y z.$
 if $\phi(y, z)$
 and $x R y$
 then $\phi(x, z)$
 then
 $\phi(x, y)$

This induction principle lets us
prove things of the form:

$x R^* y \Rightarrow \phi(x, y)$

Theorem RTC_idem:

$(R^*)^* \subseteq R^*$

Proof

By definition of $\subseteq$, it suffices
to prove

$\forall x, y \in E.$
 $x (R^*)^* y \Rightarrow x R^* y$

Now we can use induction on

$$x (R^*)^* y$$

Base case: prove $x R^* x$
 immediate from the definition

Inductive case:

assume

(i) $x R^* y$

(ii) $x R^* z$ (IH)

It follows by transitivity of R^*
 that $x R^* z$.

QED

Q4

```

|-
1 | | B
2 | |-
3 | | | ¬B
4 | | |-
5 | | | ⊥      ¬-E 1,3
6 | | ¬¬B      ¬-I 3-5
7 | B → ¬¬B    →-I 1-6

```

Here I chose Fitch-style proof
 because I like it.

You can also do proof trees,
 or tables.

Q5

Sentence: formula with no free
 variables

(a) $(0 < 0) \wedge \neg(0 < 0)$

(b) $\exists x y z.$
 $\neg(x=y) \wedge \neg(x=z) \wedge \neg(y=z)$

Here's a tempting (but wrong) answer:

$$\exists x. x \neq 0 \wedge x \neq 1$$

Q: Why is this wrong?

A: 0 and 1 might denote the same
 element.

Here's a tempting (but wrong) fix:

$$0 \neq 1 \wedge \exists x. x \neq 0 \wedge x \neq 1$$

Q: Why is this wrong?

A: It's possible for the domain to have three elements, even if 0 and 1 happen to denote the same one.

Ex: domain: {a,b,c}

model of 0: a

model of 1: a

(c)

Observation:

the reals have numbers between 0 and 1, the ints don't

$$\neg \exists x. 0 < x \wedge x < 1$$

Q6

(a)

```
x y z  →  x y z
          x z y
          y x z
          y z x
          z x y **
          z y x
```

P =

```
skip
+
swap(y,z)
+
swap(x,y)
+
swap(x,y); swap(y,z)
+
swap(x,y); swap(x,z)
+
swap(x,z)
```

(b)

```
{x = X ∧ y = Y ∧ z = Z}
P
{
  sort[x,y,z] = sort[X,Y,Z]
}
```

^ the first sketch with sets doesn't account for multiplicity

(c)

Suppose $\models \{\phi\} P \{\psi\}$

By definition of $\models$, it holds that
if $\forall s s'$.

$(s, s') \in \llbracket P \rrbracket$

and $\llbracket \phi \rrbracket s = \text{true}$, then $\llbracket \psi \rrbracket s$

Note: $\llbracket P \rrbracket$ is the union

of the semantics of every
branch of the +

$\llbracket P \rrbracket = \llbracket \text{skip} \rrbracket \cup \llbracket \text{swap}(x, y) \rrbracket$
 $\cup \dots$

$\llbracket \text{skip} \rrbracket = I$ (the identity relation)

Therefore, as a special case,
if $\forall s s'$.

$(s, s') \in \llbracket \text{skip} \rrbracket$

and $\llbracket \phi \rrbracket s = \text{true}$, then $\llbracket \psi \rrbracket s$

...and that's exactly the definition
of $\models \{\phi\} \text{skip} \{\psi\}$

Basic idea:

if a Hoare triple holds about a
program, it must hold for every
possible behaviour of that program.

A possible behaviour of P is skip .

Therefore, the triple is valid
for skip too.

General:

If $\llbracket P \rrbracket \subseteq \llbracket Q \rrbracket$

Then every triple about Q is
also valid about P .

- (d) Any Hoare triple we write
also holds about skip .
The second requirement is
not a property of skip .
Therefore, no Hoare triple
can capture the second requirement.

(e)

$\{(s, s'):$
 $\text{sort}[s(x), s(y), s(z)] =$
 $\text{sort}[s'(x), s'(y), s'(z)] \wedge$
 $s \text{ and } s' \text{ agree on all variables}$
 $\text{other than } x, y, z\}$
 $\subseteq \llbracket P \rrbracket$

$S = \text{Pow } \{q_0, q_1, q_2\}$
 $A = \{s \in S \mid s \in q_1\}$

(a)

$\{q_0\} \xrightarrow{-0} \{q_0, q_1, q_2\}$
 $\{q_0\} \xrightarrow{-1} \{\}$
 $\{q_0, q_1, q_2\} \xrightarrow{-0} \{q_0, q_1, q_2\}$
 $\{q_0, q_1, q_2\} \xrightarrow{-1} \{q_2\}$

..from those states, no further transitions are possible

(b) is just A above

$\{q_1\}$
 $\{q_0, q_1\}$
 $\{q_1, q_2\}$
 $\{q_0, q_1, q_2\}$

Q8

(a) $s(y) = 1$
 $s(z) = 2$
 $s(b) = \top$
 $s(a) = \perp$

If I use rule (3) from this state,
the invariant gets violated

(e) (a--d) asks about a safety property

The presentation here is
(deliberately) obscure
The goal is to make sure
both flags are not true.

This is a lock implementation.
Lamport's bakery algorithm.

Q: for part a could u do $(\text{swap}(x,y) + \text{swap}(y,x) + \text{swap}(x,z) ; \text{swap}(x,y) + \text{swap}(y,x) + \text{swap}(x,z))$

A: Yes.